Knowledge Transfer (KT) Document

KollegeApply Google Ads Intelligence & Operations Platform ("Command Center")

Last updated: 2026-08-13 - **Outgoing owner:** Deepak Mishra

Purpose: everything a new engineer/operator needs to run, maintain, deploy, debug and **extend** this platform without the original author. Pair with `docs/PRD.md` (the product spec).

---

1. What this system is (one paragraph)

A single FastAPI application that (a) **syncs** Google Ads data (read-only) into PostgreSQL as append-only daily snapshots, (b) serves a **React dashboard + AI ad-copy planner + approval workflow + budget governance**, and (c) sends **email** notifications. The FastAPI process **also serves the built React app** on the same port, so the whole product is one process. It never writes to Google Ads.

---

2. Tech stack

1. **Backend:** Python 3.12, FastAPI, SQLAlchemy 2.0, Alembic (migrations), pydantic-settings, APScheduler (cron jobs), `google-ads` SDK, httpx, BeautifulSoup (landing pages), openpyxl (Excel), structlog. Optional LLMs: `anthropic`, `google-generativeai` (Gemini).
1. **Frontend:** React 18 + TypeScript + Vite, TanStack Query, React Router, Tailwind CSS, Recharts, axios, lucide-react. Built to `frontend/dist` and served by FastAPI.
1. **Database:** PostgreSQL 16 (JSONB used for flexible columns).
1. **Email:** Brevo HTTP API (preferred, works on Render) or Resend, with SMTP fallback for local dev.
1. **Auth:** Google OpenID Connect (server-side OAuth), signed session cookie (stdlib HMAC — no JWT lib).
1. **Deploy:** Docker image on Render (managed Postgres). Local dev via `run_app.bat` + `docker compose` for Postgres.

---

3. Repository layout

```
app/                        # FastAPI backend
  main.py                   # app factory; serves API + frontend/dist; audit middleware
  config/settings.py        # ALL configuration (env vars) — single source of truth
  api/router.py             # mounts every endpoint router
  api/deps.py               # DI: DB session, current user, RBAC guards, filters
  api/v1/*.py               # endpoint modules (one per feature area)
  models/*.py               # SQLAlchemy models (dimensions, snapshots, app tables)
  schemas/*.py              # pydantic request/response schemas
  services/                 # business logic
    ops/*.py                # dashboards, budgets, weekly/monthly budgets, rollups
    ai/*.py                 # ad-copy generator, keyword research, landing, approval, email
    auth/*.py               # google oauth, session tokens, user provisioning
  repositories/*.py         # data-access helpers
  google_ads/*.py           # Google Ads API client + sync
  tasks/scheduler.py        # APScheduler cron jobs
alembic/versions/00XX_*.py  # migrations (numbered, chained)
frontend/src/
  pages/*.tsx               # one per route
  components/*.tsx          # shared UI (Layout/nav, tables, charts, budget editors)
```

```
  lib/queries.ts            # ALL API calls (TanStack Query hooks) — start here
  lib/types.ts              # TypeScript types mirroring API responses
  lib/api.ts                # axios instance (cookies, 401 redirect)
  auth/AuthContext.tsx      # session state via /auth/me
  state/FiltersContext.tsx  # global account + date-range filter
docs/                       # this doc, PRD, architecture, api, deploy guides
run_app.bat                 # local: build frontend + run uvicorn on :8000 (auto-restart)
Dockerfile / docker-compose.yml / render.yaml
```

Where to start reading: `frontend/src/lib/queries.ts` (what the UI calls) -> `app/api/router.py` (all endpoints) -> the relevant `app/api/v1/*.py` -> its `app/services/**`.

---

4. Local setup (run it on your machine)

1. **Prereqs:** Python 3.12, Node 18+, Docker Desktop (for Postgres).
2. **Postgres:** `docker compose up -d db` (starts Postgres on localhost:5432). Data persists in the `pgdata` volume.
3. **Python env:** `python -m venv .venv` -> activate -> `pip install -e .` (or `pip install -r requirements.txt`).
4. `.env`**:** copy `.env.example` -> `.env`, fill in `DATABASE_URL` and (optionally) Google Ads + LLM + email keys. Local `.env` is gitignored.
5. **Migrate:** `alembic upgrade head`.
6. **Frontend build:** `npm --prefix frontend install && npm --prefix frontend run build`.
7. **Run:** `run_app.bat` (Windows) — builds the frontend and runs uvicorn on **:8000** with auto-restart. Or `uvicorn app.main:app --reload`.
8. Open **http://localhost:8000**.

Tests: `pytest` (uses in-memory SQLite — no Postgres needed). ~161 tests. Run before every commit.

Typecheck/build frontend: `npm --prefix frontend run build` (runs `tsc` then Vite).

---

5. Configuration — every environment variable

All config lives in `app/config/settings.py` (class `Settings`, loaded from `.env` / real env). Never read `os.environ` elsewhere. Key groups:

1. **App:** `APP_ENV`, `APP_DEBUG`, `APP_LOG_LEVEL`, `API_PREFIX` (=/api/v1).
1. **Database:** `DATABASE_URL` (preferred) or `POSTGRES_*` parts; `DB_POOL_SIZE`, `DB_CONNECT_TIMEOUT`.
1. **Google Ads sync:** `GOOGLE_ADS_DEVELOPER_TOKEN`, `GOOGLE_ADS_CLIENT_ID`, `GOOGLE_ADS_CLIENT_SECRET`, `GOOGLE_ADS_REFRESH_TOKEN`, `GOOGLE_ADS_LOGIN_CUSTOMER_ID` (MCC id, digits only), `GOOGLE_ADS_CLIENT_CUSTOMER_IDS` (comma list, optional).
1. **Scheduler:** `SCHEDULER_ENABLED` (WARNING: **false on Render**), `SYNC_HOURLY_ENABLED`, `SYNC_DAILY_ENABLED`, `SYNC_DAILY_HOUR`, `SCORECARD_WEEKLY_ENABLED`, `WEEKLY_BUDGET_EMAIL_ENABLED`, `SYNC_DEFAULT_LOOKBACK_DAYS`.
1. **Auth (Google sign-in + RBAC):** `AUTH_ENABLED`, `GOOGLE_OAUTH_CLIENT_ID`, `GOOGLE_OAUTH_CLIENT_SECRET`, `AUTH_ALLOWED_DOMAINS` (comma, e.g. `kollegeapply.com`), `AUTH_ADMIN_EMAILS` (comma), `SESSION_SECRET` (long random), `SESSION_TTL_HOURS`, `SESSION_COOKIE_NAME`, `AUTH_REDIRECT_BASE` (optional).

1. **Email:** `BREVO_API_KEY` (preferred) or `RESEND_API_KEY`, `EMAIL_FROM` (verified sender), plus `SMTP_*` (fallback), `APPROVAL_REVIEWER_EMAIL`, `PUBLIC_BASE_URL` (used to build one-click approve links — must be the deployed URL).
1. **AI Ad Copy:** `ANTHROPIC_API_KEY` (optional), `GEMINI_API_KEY` (optional), `AD_COPY_LLM_PROVIDER` (auto|anthropic|gemini), `KEYWORD_PLANNER_ENABLED`, `LANDING_PAGE_TIMEOUT_SECONDS`.
1. **Security:** `API_KEY` (optional legacy mutation gate), `AUDIT_ENABLED`.

Rule: secrets go in `.env` (local) or the Render dashboard (prod) — **never committed**. `.env.example` holds empty placeholders only.

---

6. Auth & RBAC (how access control works)

1. **Sign-in:** `GET /api/v1/auth/google/login` -> Google consent -> `GET /api/v1/auth/google/callback` -> server verifies the id_token, provisions/updates the `User`, sets a **signed httpOnly session cookie** (HMAC via `app/services/auth/tokens.py`), redirects to `/`. `GET /auth/me` returns the current user; `POST /auth/logout` clears the cookie.
1. **Policy** (`app/services/auth/users.py`): only `AUTH_ALLOWED_DOMAINS` emails (or `AUTH_ADMIN_EMAILS`) may sign in. Admin-list emails get **admin**; everyone else is a **manager** with no accounts until assigned.
1. **Enforcement** (`app/api/deps.py`): `get_current_user` resolves the cookie. `require_admin` gates admin endpoints. `get_ops_filters` / `verify_account_access` / `verify_campaign_access` / `verify_path_account_access` **clamp every data endpoint to the user's allowed account set** — a manager literally cannot fetch another account's data. When `AUTH_ENABLED=false`, every request is a synthetic admin (login-free) so existing behavior/tests are unaffected.
1. **A manager's accounts** = the union of admin grants (`user_accounts` table, set in Users & Access) + accounts of campaigns they own (`ad_copy_generations.owner_user_id`).

To set up OAuth (one-time): create an OAuth **Web** client in Google Cloud (the "Google Auth Platform" section), Internal user type, add redirect URIs `http://localhost:8000/api/v1/auth/google/callback` and `https://<render-url>/api/v1/auth/google/callback`. Put the client id/secret + a random `SESSION_SECRET` + domains + admin emails in env, set `AUTH_ENABLED=true`.

---

7. The sync engine (getting Google Ads data in)

1. Code in `app/google_ads/` + `app/services` sync jobs. Uses the `google-ads` SDK (v24), read-only GAQL queries. Needs a **developer token (Standard access)**, an OAuth **refresh token**, and the **MCC login-customer-id**.
1. **Snapshots are append-only + idempotent** — a day's sync uses a replace-window so re-running doesn't double-count (migration `0004` removed ~11.7k historical duplicate snapshot rows).
1. **Scheduler jobs** (`app/tasks/scheduler.py`, only when `SCHEDULER_ENABLED=true`): hourly sync (top of hour), daily sync (`SYNC_DAILY_HOUR`:15), weekly scorecard (Mon 06:00), **weekly budget email (Mon 07:00)**.
1. **Reference dates:** "last N days" windows anchor to the **global freshest snapshot day across all accounts** (`app/services/ops/dates.py:resolve_ref_dates`), so a dormant account correctly shows ~0 recently instead of stale data.

---

8. Data model (tables)

Dimensions: `accounts`, `campaigns`, `ad_groups`, `ads`, `keywords`, `search_terms`, `budgets`.

Daily snapshots (append-only metrics): `campaign_snapshots` (+ device/geo variants), `ad_group_snapshots`, `ad_snapshots`, `keyword_snapshots`, `search_term_snapshots`, `budget_snapshots`.

AI / approval: `ad_copy_generations` (the plan; JSON columns for keyword_snapshot, generated_assets, scores, overrides, keyword_edits; approval_status/token; owner_user_id; submitter_user_id; ad_manager), `approval_events`, `scorecard_snapshots`.

Auth & access: `users`, `user_accounts`.

Budgets: `account_weekly_budgets`, `account_budgets` (period='month'|'total').

Ops: `alerts`, `recommendation_snapshots`, `audit_logs`, `sync_logs`, `api_tokens`.

Migrations are numbered `0001…0014` in `alembic/versions/`, each chained via `down_revision`. **To add a table/column:** create the next-numbered migration (copy an existing one's idempotent style — check-before-add), add the model in `app/models/`, and register it in `app/models/__init__.py` (import + `__all__`) so metadata + tests pick it up.

---

9. API surface (feature -> endpoints)

All under `/api/v1`. Highlights (see `app/api/v1/*.py` for the full list):

1. **Auth:** `/auth/google/login`, `/auth/google/callback`, `/auth/me`, `/auth/logout`.
1. **Dashboards:** `/dashboard/overview`, `/accounts/rollup`, `/accounts/{id}/campaigns`, `/campaigns/{id}`, `/metrics`, `/trends/*`, `/campaigns/health`, `/keywords/health`, `/searchterms/explore`, `/priorities`, `/alerts`.
1. **AI Ad Copy:** `/ai/ad-copy/generate`, `/ai/ad-copy/keyword-lookup`, `/ai/ad-copy/{id}/keywords` (edits), `/ai/ad-copy/{id}/submit|decide|approve|reject|send-approval|override|owner|ad-manager`, `/ai/ad-copy/{id}/export`, `/ai/ad-copy/landing-audit`, `/ai/ad-copy/scorecard*`, `/ai/ad-copy/portfolio`.
1. **Budgets:** `/weekly-budgets` (GET/PUT + `/email`), `/account-budget/overview` + PUT.
1. **Admin:** `/admin/users*`, `/admin/diagnostics/accounts` (data audit).

RBAC: overall account budget & downloads & user-management = admin; weekly/monthly budgets = the AM for their accounts.

---

10. Frontend map (route -> page -> data)

Routes in `frontend/src/App.tsx`; nav in `frontend/src/lib/constants.ts` (`NAV_ITEMS`, `adminOnly` flag hides admin items). Every page uses hooks from `lib/queries.ts`. Global filters (account + date range) come from `state/FiltersContext.tsx`; the top bar reads/sets them. Auth/session from `auth/AuthContext.tsx` (`useAuth()` -> `isAdmin`, `accountIds`). Key pages: `OverviewPage`, `CampaignDetailPage`, `AiAdCopyGeneratorPage` (the biggest — generation + editable keywords + approval tab), `LandingAuditorPage`, `AccountabilityPage` (+ `AccountBudgetEditor`), `WeeklyBudgetsPage` ("Budget Planner"), `AdminUsersPage` (+ account audit).

---

11. How to make common changes (recipes)

Add a new dashboard metric/column:

1. Backend: add the field in the relevant `app/services/ops/*.py` query + its pydantic schema (`app/schemas/*.py`). Response fields MUST be in the schema or FastAPI strips them.
2. Frontend: add the field to the matching interface in `lib/types.ts`, then render it in the page.

Add a new API endpoint: create/extend a module in `app/api/v1/`, include it in `app/api/router.py`, add a hook in `lib/queries.ts`. Add auth: inject `get_current_user`/`require_admin` and use the account-scope guards for anything account-specific.

Add a new page/route: create `frontend/src/pages/XxxPage.tsx`, lazy-import + `<Route>` in `App.tsx`, add to `NAV_ITEMS` (set `adminOnly` if needed).

Add a scheduled job: add an `add_job(...)` in `app/tasks/scheduler.py` guarded by a setting; write the job in `app/services/**` opening its own `session_scope()`.

Change the AI ad-copy logic: `app/services/ai/ad_copy_service.py` (orchestration) + `keyword_research_service.py` (Planner/keywords) + `budget_planner.py` (forecast) + `landing_*` + `approval_service.py` (email/approval). The generator is provider-agnostic (deterministic fallback if no LLM key).

Always: run `pytest`, `npm --prefix frontend run build`, then commit. **Commit as `DeepakMishra01`; do NOT add "Co-Authored-By: Claude".** Commit or push only when asked; branch off main for risky changes.

---

12. Deployment (Render) — read carefully

1. **Model:** the **local machine is the source of truth** (runs the scheduler/sync). **Render is a read-only mirror** with `SCHEDULER_ENABLED=false` (pinned in `render.yaml`) — it does **not** sync live. Fresh data is pushed by `pg_dump` (local) -> `pg_restore` (Render). App runs from the Docker image; migrations run on deploy.
1. **Render URL:** `https://ads-intelligence-mnr2.onrender.com`. Deploys on push to `main` (GitHub `DeepakMishra01/google-ads-intelligence`).
1. **Env on Render:** set all auth/email/DB vars in the Render dashboard Environment tab (never in git). After changing env, Render redeploys.
1. **Consequence:** the automatic Monday budget email won't fire on Render (scheduler off). Use the "Email admins now" button, or add a Render Cron Job hitting `POST /api/v1/weekly-budgets/email`, or enable the scheduler on Render.
1. Full steps: `docs/deploy-render.md`.

---

13. Email setup (Brevo — no DNS needed to start)

Render blocks outbound SMTP, so email goes over **HTTPS** via **Brevo** (or Resend). Setup: create a Brevo account -> **Senders** -> add & verify a sender by clicking the emailed link (no DNS) -> **SMTP & API -> API Keys** -> generate -> set `BREVO_API_KEY` + `EMAIL_FROM` on Render. Dispatch order in `app/services/ai/email_service.py`: Brevo -> Resend -> SMTP. Without DNS (SPF/DKIM) mail may land in spam initially (mark "not spam"); add DNS later for inbox delivery.

---

14. Known issues & gotchas (must-read)

1. **Duplicate account/campaign records:** the DB has duplicate campaign (and some account) dimension rows from historical double-imports, plus snapshotdimension id misalignment (a campaign's stored name may not

match its metrics). The **Account data audit** (Users & Access -> "Account data audit") flags duplicate customer_ids + shows each account's latest data date. Proper fix = a source-level dedup migration (keep one row per (account, google id), realign snapshot FKs), then re-mirror. Not yet done.

2. **Stale/dormant accounts:** an account inactive for months has old snapshots; the window anchoring (§7) handles this so "last 30 days" reads ~0 not stale.
3. **Render doesn't sync** (§12).
4. **Conversion tracking absent on many accounts** -> leads show 0 (honest, not a bug).
5. **Pmax/Demand Gen** have no keywords/search terms.
6. **Line endings:** git warns LF->CRLF on Windows; harmless.

---

15. Secrets to rotate at handover (do this)

These were exposed during development and should be rotated by the new owner:

1. Google **OAuth client secret** (Google Cloud -> Credentials -> rotate).
1. **SESSION_SECRET** (generate a new random; rotating just logs everyone out).
1. **Brevo/Resend API key**.
1. Render **Postgres password**.
1. **Gemini API key** (was leaked in `.env.example` git history) — rotate in Google AI Studio.

Update the new values only in `.env` (local) and Render env — never commit.

---

16. Troubleshooting runbook

Symptom	Likely cause	Fix
Dashboard shows old/zero data	Render is a stale mirror / dormant account	Re-mirror from local; check Account data audit
"Couldn't load … is the database running?"	Postgres down	`docker compose up -d db` (local)
Approval email "not sent: timed out / network unreachable"	Render blocks SMTP	Use Brevo (`BREVO_API_KEY`) — §13
Budget edit "save failed"	403 (not admin / not your account) or DB	Check role/assignment; watch the per-cell status
Manager sees nothing	No accounts assigned	Assign in Users & Access (or set as campaign owner)
"All time" filter errors	(fixed) days cap	Ensure endpoints allow `le=3650`
Numbers differ from Google Ads	Data freshness / window / duplicates	§7, §14; run a fresh sync
Keywords irrelevant for a college	Ambiguous brand name	Type the full official name; off-topic filter is in `keyword_research_service.py`

Logs: structlog to stdout (Render logs tab). **Health:** `GET /api/v1/health`. **Sync status:** shown on Overview + `sync_logs` table.

---

17. Handover checklist

- [ ] Rotate all secrets (§15).
- [ ] Confirm you can sign in as admin on Render; add your email to `AUTH_ADMIN_EMAILS`.
- [ ] Confirm local dev runs (Postgres up, migrate, build, run).
- [ ] Do one full local **sync**, then a `pg_dump`/`pg_restore` to Render (re-mirror).
- [ ] Set Brevo email + test the approval + Monday budget emails.
- [ ] Read `docs/PRD.md`, `docs/architecture.md`, `docs/deploy-render.md`, `docs/ai_ad_copy/`.
- [ ] Plan the duplicate-account dedup (§14.1) — the biggest open data-quality item.

Companion: `docs/PRD.md`. Architecture deep-dive: `docs/architecture.md`. API: `docs/api.md`. ERD: `docs/er-diagram.md`.